

Wake Peak-Seeking: Architecture & Integration Points

geoFS_pilot -- how a custom wake model and an extremum-seeking controller plug into the existing stack

1. What already exists

The aircraft/guidance stack is already layered so each piece is swappable without touching the others: Aircraft (JSBSimAircraft) <- WindField <- Navigator/OffsetNavigator <- Autopilot <- Simulation/run_sim*.py. The wake itself is a WindField (src/atmosphere/wind_field.py):

```
class WindField(ABC):
    def wind_ned(self, x, y, z, t) -> np.ndarray: # [Vn, Ve, Vd] m/s, NED
        ...
```

x=north(m), y=east(m), z=down(m), t=time(s). LambOseenVortexPairWake already implements this for a trailing-vortex pair. A custom wake as a function of (x, y, t) is a drop-in: write a new WindField subclass (anywhere, e.g. src/atmosphere/wind_field.py or your own module), register its config-driven construction in src/atmosphere/factory.py's build_wind_field(wake_config), and add a matching block to config/wake.yaml. Nothing in Aircraft, Navigator, or Autopilot needs to change -- that boundary already exists and is exercised.

2. The missing piece: a measurable quantity, not the wake itself

The real aircraft cannot query wind_ned() directly -- it can only feel the wake's aerodynamic effect on itself (induced drag, lift asymmetry, required thrust, etc.). To keep the peak-seeking controller honest, it must receive only a scalar reading, never the WindField. Introduce a small sensor interface, e.g. src/guidance/wake_sensor.py:

```
class WakeQuantitySensor(ABC):
    def measure(self, state: AircraftState, wind_field: WindField, t: float) -> float:
        ... # e.g. an induced-drag/CD proxy
```

The sensor is allowed to depend on WindField (it models a real aerodynamic sensor's dependence on local airflow -- e.g. computing CD via the already-existing Aircraft.get_cl_cd(wind_field)); the discipline is that the WindField reference stops here. Whatever consumes the sensor's output must see only the float.

3. New Navigator: extremum-seeking as a composition, not a rewrite

Add src/guidance/extremum_seeking_navigator.py, ExtremumSeekingNavigator, implementing the same compute(state, quantity) -> GuidanceCommand contract (one extra argument vs. today's compute(state)). It should not reimplement the lateral guidance law -- it should wrap an internal OffsetNavigator instance and dynamically retarget its target_lateral_m each step, reusing the already-validated pure-pursuit bearing law instead of duplicating it:

```
dither = A * sin(omega * t)
gradient_est += lpf( quantity * sin(omega * t) ) # demodulate
lateral_estimate -= k * gradient_est * dt # descend toward the minimum
self._inner_offset_nav.target_lateral_m = lateral_estimate + dither
return self._inner_offset_nav.compute(state)
```

As the real wake drifts, the measured quantity's minimum moves with it, the gradient estimate re-biases, and the aircraft tracks it -- all without ever reading wind_field.wind_ned() from the controller side.

4. Wiring

- config/simulation.yaml: add flightgear.guidance_mode = "extremum_seeking", plus a guidance.extremum_seeking block (dither amplitude/frequency, gain, low-pass time constant, initial lateral_estimate_m) -- same pattern as today's guidance.lateral_offset.
- scripts/run_sim.py / run_sim_flightgear.py / dashboard.py: construct the WakeQuantitySensor alongside the existing build_wind_field(wake_config) call; in the step loop, compute quantity = sensor.measure(state, wind_field, t) and pass it into navigator.compute(state, quantity) only for this mode -- the same branch point that already selects Navigator vs. OffsetNavigator via guidance_mode.
- src/sim/logger.py: TelemetryRow already has spare room conceptually -- add measured_quantity_m and lateral_estimate_m fields so the dashboard's existing 7-panel layout can plot the seeking behavior live (a new 8th panel, or replace one of the less critical ones).
- Everything else -- Aircraft, Autopilot, Simulation, FlightGear streaming -- is untouched.

5. Net-new files vs. touched files

New: src/guidance/wake_sensor.py, src/guidance/extremum_seeking_navigator.py, one WindField subclass for the custom wake (wherever convenient). Touched (config/wiring only, few lines each): src/atmosphere/factory.py, config/wake.yaml, config/simulation.yaml, the run_sim*/dashboard entry points. This matches the README's existing roadmap note on wake minimum-seeking and keeps Aircraft/Autopilot/dynamics untouched, as designed.